LAB REPORT: CPU SCHEDULING ALGORITHMS

Course: Operating Systems

Student Name: Aryan Nautiyal **Roll Number:** 251210015

Aim

To implement and simulate CPU scheduling algorithms—Highest Response Ratio Next and Round Robin —using process arrival and burst times, and to evaluate and compare key performance metrics.

Theory

Metric	Formula / Definition
Completion Time (CT)	Time when process finishes execution
Turnaround Time (TAT)	$\mathrm{TAT} = \mathrm{CT} - \mathrm{AT}$
Waiting Time (WT)	$\mathrm{WT} = \mathrm{TAT} - \mathrm{BT}$
Response Time (RT)	$\mathrm{RT} = \mathrm{First\ Start\ Time} - \mathrm{AT}$

- **HRRN (Non-Preemptive):** Selects process with highest Response Ratio $\mathrm{RR} = \frac{\mathrm{WT} + \mathrm{BT}}{\mathrm{BT}}$. Prevents starvation of long processes while favoring short ones.
- **Round Robin (Preemptive):** Allocates CPU for a fixed time quantum. Once quantum expires, process is moved to ready queue rear.

Source Code (C++)

```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>
#include <iomanip>
#include <climits>

using namespace std;

struct Process {
    int id, at, bt, remaining_bt;
    int ct = 0, tat = 0, wt = 0, rt = -1;
    Process(int id, int at, int bt) : id(id), at(at), bt(bt), remaining_bt(bt) {}
};

void printReport(const vector<Process> &procs, const string &algo_name) {
    cout << " Algorithm: " << algo_name << "\n";
    cout << left << setw(6) << "PID" << setw(6) << "AT" << setw(6) << "BT"
         << setw(6) << "CT" << setw(6) << "TAT" << setw(6) << "WT" << setw(6) << "RT" << "\n";
    double total_tat = 0, total_wt = 0, total_rt = 0;
    for (const auto &p : procs) {
        cout << left << setw(6) << ("P" + to_string(p.id)) << setw(6) << p.at
             << setw(6) << p.bt << setw(6) << p.ct << setw(6) << p.tat
             << setw(6) << p.wt << setw(6) << p.rt << "\n";
        total_tat += p.tat; total_wt += p.wt; total_rt += p.rt;
    }
    int n = procs.size();
    cout << fixed << setprecision(2);
    cout << "\nAverage TAT : " << (total_tat / n)
         << "\nAverage WT  : " << (total_wt / n)
         << "\nAverage RT  : " << (total_rt / n) << "\n\n";
}

void calculateHRRN(vector<Process> procs) {
    sort(procs.begin(), procs.end(), [](const Process &a, const Process &b) { return a.at < b.at; });
    int current_time = 0, completed = 0, n = procs.size();
    vector<bool> is_completed(n, false);
    while (completed < n) {
        int best_idx = -1; double max_rr = -1.0;
        for (int i = 0; i < n; i++) {
            if (!is_completed[i] && procs[i].at <= current_time) {
                double rr = (double)((current_time - procs[i].at) + procs[i].bt) / procs[i].bt;
                if (rr > max_rr) { max_rr = rr; best_idx = i; }
            }
        }
        if (best_idx == -1) {
            int next_at = INT_MAX;
            for (int i = 0; i < n; i++) if (!is_completed[i]) next_at = min(next_at, procs[i].at);
            current_time = next_at;
        } else {
            procs[best_idx].wt = procs[best_idx].rt = current_time - procs[best_idx].at;
            procs[best_idx].ct = current_time + procs[best_idx].bt;
```

```
            procs[best_idx].tat = procs[best_idx].ct - procs[best_idx].at;
            current_time += procs[best_idx].bt;
            is_completed[best_idx] = true; completed++;
        }
    }
    sort(procs.begin(), procs.end(), [](const Process &a, const Process &b) { return a.id < b.id; });
    printReport(procs, "Highest Response Ratio Next (HRRN)");
}

void calculateRoundRobin(vector<Process> procs, int quantum) {
    sort(procs.begin(), procs.end(), [](const Process &a, const Process &b) { return a.at < b.at; });
    int current_time = 0, completed = 0, n = procs.size(), arrival_idx = 0;
    queue<int> ready_q;
    while (completed < n) {
        while (arrival_idx < n && procs[arrival_idx].at <= current_time) ready_q.push(arrival_idx++);
        if (!ready_q.empty()) {
            int curr = ready_q.front(); ready_q.pop();
            if (procs[curr].rt == -1) procs[curr].rt = current_time - procs[curr].at;
            int time_slice = min(quantum, procs[curr].remaining_bt);
            current_time += time_slice; procs[curr].remaining_bt -= time_slice;
            while (arrival_idx < n && procs[arrival_idx].at <= current_time) ready_q.push(arrival_idx++);
            if (procs[curr].remaining_bt > 0) ready_q.push(curr);
            else {
                procs[curr].ct = current_time;
                procs[curr].tat = procs[curr].ct - procs[curr].at;
                procs[curr].wt = procs[curr].tat - procs[curr].bt;
                completed++;
            }
        } else { current_time++; }
    }
    sort(procs.begin(), procs.end(), [](const Process &a, const Process &b) { return a.id < b.id; });
    printReport(procs, "Round Robin (Q = " + to_string(quantum) + ")");
}

int main() {
    vector<Process> processes = {
        Process(1, 0, 8), Process(2, 1, 4), Process(3, 2, 2),
        Process(4, 3, 1), Process(5, 4, 3)
    };
    calculateHRRN(processes);
    calculateRoundRobin(processes, 3);
    return 0;
}
```

Terminal Output

```
 Algorithm: Highest Response Ratio Next (HRRN)
PID   AT    BT    CT    TAT   WT    RT
P1    0     8     8     8     0     0
P2    1     4     15    14    10    10
P3    2     2     11    9     7     7
P4    3     1     9     6     5     5
P5    4     3     18    14    11    11

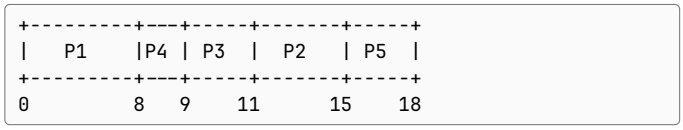
Average TAT : 10.20
Average WT  : 6.60
Average RT  : 6.60

 Algorithm: Round Robin (Q = 3)
PID   AT    BT    CT    TAT   WT    RT
P1    0     8     18    18    10    0
P2    1     4     16    15    11    2
P3    2     2     8     6     4     4
P4    3     1     9     6     5     5
P5    4     3     15    11    8     8

Average TAT : 11.20
Average WT  : 7.60
Average RT  : 3.80
```

Gantt Charts

HRRN:



Round Robin ($Q = 3\text{ms}$):

